BACKEND ENGINEERING

DAY 5

Relational Database Fundamentals & SQL Basics

Practical + Research-Oriented Learning

Learn → Understand → Set Up → Explore → Build → Test → Explain

Your starting point: You have learned Python fundamentals, FastAPI routes, path/query parameters, request bodies, Pydantic validation, error handling, project structure and an introduction to MongoDB CRUD. Day 5 follows the original learning roadmap: understand relational databases and SQL before going deeper into PostgreSQL integration and ORM usage.

Today's goal: understand how relational databases organise data, how tables are connected, and how to write basic SQL queries to create, insert, retrieve, filter, update and delete records using a Hospital ERP example.

Suggested time: 2.5–3.5 hours | **Level:** Beginner backend developer

Learning rule: predict what each query will do before running it. Understand which rows are affected and why.

How Day 5 fits your learning path

Day 1 introduced backend concepts and the client/server model. Day 2 introduced FastAPI routes and request data. Day 3 strengthened validation and error responses. Day 4 organised the application and introduced MongoDB persistence. Day 5 now builds the relational database foundation described in the original Day 1 roadmap.

Previous learning	What it contributes today
FastAPI routes	An endpoint will eventually call SQL queries or database operations.
Pydantic models	Validate API input before saving it to a relational database.
MongoDB documents	Compare document storage with relational tables.
Python dictionaries and lists	Understand how query results become Python objects.
Exceptions and HTTP errors	Translate missing records or conflicts into meaningful API responses.

What you will learn

• What a relational database is and why tables are used.

• Rows, columns, data types, primary keys and foreign keys.

• How to design related Hospital ERP tables.

• Basic SQL commands: CREATE, INSERT, SELECT, WHERE, ORDER BY, UPDATE and DELETE.

• How to combine related tables using JOIN.

• Why constraints and relationships protect data quality.

• How SQL differs from MongoDB queries and documents.

• How to practise using PostgreSQL or a local SQL environment.

Today's vocabulary

Term	Simple meaning	Hospital ERP example
Database	Organised storage for related data	hospital_erp
Table	A collection of rows with defined columns	patients
Row / record	One stored entry	One patient
Column / field	One attribute of each entry	patient_name
Primary key	Unique identifier for a row	patient_id
Foreign key	A value linking to another table	department_id
SQL	Language used to work with relational databases	SELECT patient_name FROM patients
Constraint	A rule enforced by the database	NOT NULL, UNIQUE

1. What is a relational database?

A relational database stores data in tables. A table has columns that describe the type of information and rows that contain individual records. Tables can be connected through keys. This structure is useful when information has clear relationships—for example, a patient belongs to a department, and an appointment connects a patient with a doctor.

Example: patients table

patient_id	patient_name	age	department_id
101	Ravi Kumar	35	10
102	Meena Rao	28	20
103	Arjun Das	42	10

Each row represents one patient. patient_id identifies the patient. department_id indicates which department the patient is associated with.

2. Why not store everything in one large table?

Imagine storing patient details, doctor details, department names and appointment details repeatedly in one table. The same doctor or department information may be copied many times. This creates duplication and makes updates difficult.

A relational design separates related information into tables and connects them using keys.

departments table	doctors table
department_id | department_name	doctor_id | doctor_name | department_id
10 | Cardiology	501 | Dr. Rao | 10
20 | Neurology	502 | Dr. Meena | 20

The department_id in doctors refers to the department_id in departments. This is a relationship between two tables.

3. Primary key

A primary key uniquely identifies each row in a table. It must be unique and cannot be NULL. A table normally has one primary key, which can contain one or more columns.

```
CREATE TABLE departments (
    department_id INTEGER PRIMARY KEY,
    department_name VARCHAR(80) NOT NULL
);
```

Here department_id is the primary key. Two departments cannot have the same primary-key value.

4. Foreign key

A foreign key creates a relationship by referring to a key in another table. It helps prevent records from referring to a department that does not exist.

```
CREATE TABLE doctors (
    doctor_id INTEGER PRIMARY KEY,
    doctor_name VARCHAR(100) NOT NULL,
    department_id INTEGER,
    FOREIGN KEY (department_id)
        REFERENCES departments(department_id)
);
```

The doctors table can refer to a valid department row. The database can reject an invalid department_id when the foreign-key constraint is enforced.

Architect thinking: keys are not just ID columns. They define identity and relationships, and they help the database protect consistency.

5. Set up a practice database

You can practise SQL using PostgreSQL, which is the relational database option in your learning path. Install PostgreSQL locally or use a trusted managed PostgreSQL service. Create a practice database named hospital_erp. If PostgreSQL is not ready yet, use an SQL playground for the query exercises, but remember that a playground may not preserve your work.

Create the database

```
CREATE DATABASE hospital_erp;
```

Connect your SQL client to hospital_erp before creating the tables. In a graphical client, choose the database from the connection/browser panel. In psql, connect with a command such as \c hospital_erp.

6. SQL data types — common beginner types

Type	Purpose	Example
INTEGER	Whole numbers	age, department_id
VARCHAR(n)	Text with a maximum length	patient_name VARCHAR(100)
TEXT	Text without a declared small maximum	notes
DATE	Calendar date	appointment_date
TIMESTAMP	Date and time	created_at
BOOLEAN	True or false	is_active
NUMERIC(10,2)	Exact decimal amount	invoice_total

Choose data types according to the meaning of the data. For example, do not store a date as an arbitrary string if you need to filter or sort it as a date.

7. CREATE TABLE — define the structure

```
CREATE TABLE patients (
    patient_id INTEGER PRIMARY KEY,
    patient_name VARCHAR(100) NOT NULL,
    age INTEGER CHECK (age >= 0 AND age <= 120),
    phone VARCHAR(20) UNIQUE,
    department_id INTEGER,
    FOREIGN KEY (department_id)
        REFERENCES departments(department_id)
);
```

Read it line by line:

• CREATE TABLE patients creates a new table named patients.

• patient_id INTEGER PRIMARY KEY stores a unique whole-number identifier.

• patient_name VARCHAR(100) NOT NULL requires a name and limits its length.

• CHECK constrains age to a sensible range for this exercise.

• UNIQUE prevents duplicate phone values when a value is supplied.

• FOREIGN KEY connects department_id to the departments table.

Note: this example assumes the departments table has already been created. Create referenced tables before tables that depend on them.

8. INSERT — add records

```
INSERT INTO departments (department_id, department_name)
VALUES
    (10, 'Cardiology'),
    (20, 'Neurology'),
    (30, 'Orthopedics');

INSERT INTO patients
    (patient_id, patient_name, age, phone, department_id)
VALUES
    (101, 'Ravi Kumar', 35, '9000000001', 10),
    (102, 'Meena Rao', 28, '9000000002', 20),
    (103, 'Arjun Das', 42, '9000000003', 10);
```

INSERT INTO names the table and columns. VALUES supplies the values in the same order as the listed columns. Explicitly listing columns makes the query easier to read and safer if the table structure changes.

9. SELECT — read data from a table

```
-- Read every column and every row
SELECT * FROM patients;

-- Read only selected columns
SELECT patient_id, patient_name, age
FROM patients;
```

SELECT chooses the columns you want to see. FROM identifies the table. The asterisk means all columns; in application code, prefer selecting only the fields you need.

10. WHERE — filter rows

```
-- Find one patient by ID
SELECT *
FROM patients
WHERE patient_id = 101;

-- Find patients older than 30
SELECT patient_name, age
FROM patients
WHERE age > 30;

-- Find patients in department 10
SELECT *
FROM patients
WHERE department_id = 10;
```

WHERE restricts which rows are returned. Comparison operators include =, != or <>, >, <, >= and <=.

11. AND, OR and IN

```
-- Both conditions must be true
SELECT *
FROM patients
WHERE age > 30 AND department_id = 10;

-- Either condition can be true
SELECT *
FROM patients
WHERE department_id = 10 OR department_id = 20;

-- Match any value in a list
SELECT *
FROM patients
WHERE department_id IN (10, 20);
```

Use parentheses when combining AND and OR in a complex condition so the intended logic is clear.

12. LIKE — search text patterns

```
-- Names starting with R
SELECT *
FROM patients
WHERE patient_name LIKE 'R%';

-- Names containing 'a'
SELECT *
FROM patients
WHERE patient_name LIKE '%a%';
```

In PostgreSQL, LIKE is case-sensitive for ordinary text matching. ILIKE performs case-insensitive matching. % means any sequence of characters; _ means one character.

13. ORDER BY and LIMIT

```
-- Oldest patients first
SELECT patient_name, age
FROM patients
ORDER BY age DESC;

-- Youngest patients first; show only two
SELECT patient_name, age
FROM patients
ORDER BY age ASC
LIMIT 2;
```

ASC sorts ascending; DESC sorts descending. LIMIT restricts the number of rows returned. Always use a stable ordering when building paginated APIs.

14. UPDATE — change existing records

```
UPDATE patients
SET age = 36
WHERE patient_id = 101;
```

UPDATE identifies the table, SET defines the new values, and WHERE selects the rows to change.

Warning: an UPDATE without a WHERE clause changes every row in the table. Before running an update, first run a SELECT with the same WHERE condition to confirm which rows will be affected.

15. DELETE — remove records

```
DELETE FROM patients
WHERE patient_id = 103;
```

DELETE removes rows. Without a WHERE clause, it deletes all rows from the table. Be especially careful when running DELETE against real data.

16. NULL — missing or unknown values

```
-- Find patients whose department is not set
SELECT *
FROM patients
WHERE department_id IS NULL;

-- Find patients whose department is set
SELECT *
FROM patients
WHERE department_id IS NOT NULL;
```

NULL does not mean zero or an empty string. It represents missing or unknown data. Use IS NULL and IS NOT NULL rather than = NULL.

17. Aggregate functions

```
-- Count patients
SELECT COUNT(*) AS total_patients
FROM patients;

-- Average patient age
SELECT AVG(age) AS average_age
FROM patients;

-- Highest and lowest age
SELECT MAX(age) AS oldest_age,
       MIN(age) AS youngest_age
FROM patients;
```

Aggregate functions calculate a result across multiple rows. Common examples are COUNT, SUM, AVG, MIN and MAX.

18. GROUP BY — summarise by category

```
SELECT department_id, COUNT(*) AS patient_count
FROM patients
GROUP BY department_id;
```

GROUP BY combines rows with the same department_id into groups. COUNT(*) counts how many patient records are in each group.

19. JOIN — read related tables together

A JOIN combines rows from two or more tables using a relationship. In the Hospital ERP, a patient row stores department_id; the departments table stores the department name.

```
SELECT
    p.patient_id,
    p.patient_name,
    d.department_name
FROM patients AS p
JOIN departments AS d
    ON p.department_id = d.department_id;
```

AS gives tables short aliases. ON states how the rows match. An INNER JOIN (written JOIN) returns rows with matching values in both tables.

20. LEFT JOIN — keep all rows from the left table

```
SELECT
    d.department_name,
    p.patient_name
FROM departments AS d
LEFT JOIN patients AS p
    ON d.department_id = p.department_id;
```

A LEFT JOIN returns every department, even when no patient matches it. The patient columns are NULL when there is no matching patient.

JOIN type	What it returns
INNER JOIN	Only rows with matches on both sides.
LEFT JOIN	All rows from the left table, plus matching rows from the right.
RIGHT JOIN	All rows from the right table, plus matching rows from the left.
FULL OUTER JOIN	All rows from both sides; unmatched columns are NULL.

21. A simple relationship diagram

```
departments
-----------
department_id  PRIMARY KEY
department_name

       1
       |
       | one department can have many doctors
       |
       +--------------------< doctors
                              -------
                              doctor_id PRIMARY KEY
                              doctor_name
                              department_id FOREIGN KEY

patients
--------
patient_id PRIMARY KEY
patient_name
department_id FOREIGN KEY
```

The exact relationship rules depend on the application. A real Hospital ERP might model patient-to-department assignment differently from patient-to-appointment relationships. Design based on the business meaning.

22. MongoDB vs SQL — connect Day 4 and Day 5

Concept	MongoDB	PostgreSQL
Storage	Collections of documents	Tables of rows
Record	Document	Row
Attribute	Field	Column
Identifier	Often _id / ObjectId	Primary key column
Relationship	Embedded data or references	Foreign keys and joins
Read example	find_one({"_id": ...})	SELECT ... WHERE patient_id = ...
Update example	update_one(..., {"$set": ...})	UPDATE ... SET ... WHERE ...
Schema	Flexible document shape; application can enforce rules	Defined table structure and constraints

Both approaches can store persistent data. The important difference is how data is organised and related. MongoDB remains your main database practice path from Day 4; Day 5 introduces SQL because your original learning roadmap places relational fundamentals before deeper PostgreSQL/ORM work.

23. How SQL will connect to FastAPI later

Today, practise SQL independently first. In the next relational database lesson, you will connect FastAPI to PostgreSQL through a Python database library and learn how sessions, models and transactions work. A typical request flow will be:

```
Client sends HTTP request
        |
        v
FastAPI route receives request
        |
        v
Pydantic validates request body
        |
        v
Route / service uses database session
        |
        v
SQL query runs against PostgreSQL
        |
        v
Rows are converted into response data
        |
        v
FastAPI sends JSON response
```

FastAPI is the API framework; PostgreSQL stores relational data; SQL is the language used to query it; a Python driver/ORM helps your Python application communicate with the database.

24. Common beginner mistakes

Mistake	Better habit
Forgetting WHERE in UPDATE or DELETE	Preview affected rows with SELECT first.
Confusing primary and foreign keys	Primary key identifies a row; foreign key references another table.
Using = NULL	Use IS NULL or IS NOT NULL.
Assuming JOIN means every row is returned	Know whether you need INNER, LEFT, RIGHT or FULL JOIN.
Using SELECT * everywhere	Select only the columns required.
Storing all related information in one table	Separate entities and connect them with keys where appropriate.
Assuming SQL is Python	SQL is a database query language; Python calls it through a driver or ORM.

25. Practical lab — build the Hospital ERP database

Complete these tasks in your PostgreSQL practice database. Run one query at a time and inspect the result.

1. Create the departments table with department_id as the primary key and department_name as NOT NULL and UNIQUE.

2. Insert three departments: Cardiology, Neurology and Orthopedics.

3. Create the doctors table with doctor_id as primary key, doctor_name as NOT NULL and department_id as a foreign key.

4. Insert at least three doctors across the departments.

5. Create the patients table with patient_id as primary key, patient_name as NOT NULL, age with a CHECK constraint and department_id as a foreign key.

6. Insert at least five sample patients. Use fictional names and contact information.

7. Display all patients using SELECT.

8. Display only patient_name and age for patients older than 30.

9. Display patients in Cardiology using department_id.

10. Sort patients by age from highest to lowest.

11. Count how many patients are in each department using GROUP BY.

12. Use JOIN to show each patient's name and department name.

13. Use LEFT JOIN to list all departments, including departments with no patients.

14. Update one patient's age after checking the target row with SELECT.

15. Delete one practice patient after confirming the patient_id.

26. Practice questions

Q1. Explain the difference between a database, table, row and column.
Answer: _____

Q2. What is a primary key? Why should it be unique?
Answer: _____

Q3. What is a foreign key? Give a Hospital ERP example.
Answer: _____

Q4. What is the purpose of NOT NULL and UNIQUE?
Answer: _____

Q5. What does SELECT * FROM patients do?
Answer: _____

Q6. What is the role of WHERE?
Answer: _____

Q7. What is the difference between AND and OR?
Answer: _____

Q8. What does ORDER BY age DESC do?
Answer: _____

Q9. What is the danger of UPDATE without WHERE?
Answer: _____

Q10. Why do we use IS NULL instead of = NULL?
Answer: _____

Q11. What is the purpose of GROUP BY?
Answer: _____

Q12. What is the difference between INNER JOIN and LEFT JOIN?
Answer: _____

Q13. How is a MongoDB document different from a PostgreSQL row?
Answer: _____

Q14. What role does SQL play when a FastAPI route uses PostgreSQL?
Answer: _____

27. Day 5 assignment

• Create the three tables: departments, doctors and patients.

• Insert at least three departments, three doctors and five patients.

• Write ten SELECT queries using WHERE, IN, LIKE, ORDER BY and LIMIT.

• Write two aggregate queries using COUNT and AVG.

• Write one INNER JOIN and one LEFT JOIN.

• Demonstrate one safe UPDATE and one safe DELETE on practice data.

• Draw the tables and mark primary-key and foreign-key columns.

• Write five lines comparing relational tables with MongoDB documents.

Day 5 completion checklist

• I can explain what a relational database is.

• I understand tables, rows, columns and common SQL data types.

• I can identify primary keys and foreign keys.

• I can create tables with basic constraints.

• I can insert and retrieve records.

• I can filter, sort and limit query results.

• I can update and delete records safely.

• I can use COUNT, AVG and GROUP BY.

• I understand INNER JOIN and LEFT JOIN.

• I can compare relational tables with MongoDB documents.

• I understand how SQL will fit into a future FastAPI + PostgreSQL connection.

Official documentation — primary references

• PostgreSQL — Tutorial
https://www.postgresql.org/docs/current/tutorial.html

• PostgreSQL — SQL Commands
https://www.postgresql.org/docs/current/sql-commands.html

• PostgreSQL — Data Types
https://www.postgresql.org/docs/current/datatype.html

• PostgreSQL — Constraints
https://www.postgresql.org/docs/current/ddl-constraints.html

• PostgreSQL — Queries / SELECT
https://www.postgresql.org/docs/current/queries.html

• PostgreSQL — Joins
https://www.postgresql.org/docs/current/tutorial-join.html

• FastAPI — SQL Databases
https://fastapi.tiangolo.com/tutorial/sql-databases/

• SQLModel — Documentation
https://sqlmodel.tiangolo.com/

Today's success is not “I memorised SQL.” It is “I can explain how relational data is organised, write queries to retrieve and change it, and understand how tables connect through keys.”